

TCLK → ACLK → ACLK-Lite Pipeline

Single-board timing-link loop: decode TCLK, White-Rabbit-timestamp it, re-encode as gigabit ACLK over SFP, decode back on a shared WR timeline, mirror as ACLK-Lite, and publish every event to Redis Streams

Subsystem	<code>aclk_pipeline_bd_top</code> (rtl/aclk_pipeline_bd_top.v) + board-side Python
Audience	An engineer integrating this bitstream, or consuming its events from Redis
Target / board	AMD Kria KR260 (Zynq UltraScale+ <code>xck26-sfvc784-2LV-c</code>)
Source revision	branch <code>redis-convention</code> @ <code>d87b3c0</code>
Status	Grounded in repository source; see Source Traceability appendix

1. Purpose and Scope

`aclk_pipeline_bd_top` is a single-board, single-bitstream pipeline that closes the full timing-link loop on one KR260 and hands every event to a board-side Redis publisher. It receives Fermilab **TCLK** on a 3.3V logic pin, decodes and **White-Rabbit-timestamps** every event and publishes it to the PS over AXI4-Lite; re-encodes that same event stream as a **gigabit ACLK** 8b10b word and transmits it over the SFP+ optical transceiver; receives the ACLK back on the **same board** through a physical fiber loopback, decodes and timestamps it against the **same WR timeline**, and publishes it to the PS over a **second** AXI4-Lite node; re-encodes the decoded-back events as **ACLK-Lite** (Manchester biphase-mark) on a Pmod pin as a scope probe; and exposes the WR timebase itself on a **third** AXI4-Lite node for arming and health. On the PS, one Python publisher per readout drains the FIFO and writes each event into **local Redis Streams** under the `KR260:` namespace.

Both readouts stamp events from one White-Rabbit-disciplined `{sec, ns}` timeline, so `ts(ACLK-back) - ts(TCLK-in)` for a matched event is the true H12 → decode → encode → SFP → decode round-trip latency in real nanoseconds, and every published event carries an absolute UTC time that correlates across boards.

In scope: the external interface of the integrated top (pins, clocks, resets, the **three** AXI4-Lite slaves and their register maps), the WR timebase bring-up, the data-flow contract, GT/SFP recovery behavior, the PS-side read flow, and the **board-side Redis publisher and its key schema**.

Out of scope: two-board operation against a live Fermilab ACLK fiber, SFP DDM/I2C telemetry, and any Redis *consumer* (this design is publish-side only) (deploy/redis.md:4).

2. System Overview

The design has two tiers: the **PL bitstream** (the timing loop and its three AXI slaves) and the **PS software** (the readers, the WR arm tool, and the Redis publishers). Events flow left-to-right through the PL, then up into Redis.

2.1 PL data path (one bitstream, one SFP fiber looped TX → RX)

```
WR ref (E10 10MHz, E12 PPS) ==> wr_timebase x3 ==> shared {sec,ns} timeline
                                |
                                (S_AXI3 @ 0x8002_0000)
H12 ==> tclk_readout_top ==> S_AXI @ 0x8000_0000 ==> /dev/uio (tclk)
(biphase) | (TCLK_RCV, clk_40m; ts <- ts_tclk)
          |
          +== dbg_data/dbg_dav ==> aclk_tclk_encoder ==> GT TX ==> SFP TX
              (clk_40m)          (tx_usrclk2 62.5M; count RAM, CRC8, 96->16) [fiber loop]
                                  |
                                  rx_data16/K <== GT RX 8b10b + comma align <== SFP RX <
                                  | (rx_usrclk2 62.5M)
                                  v
aclk_gt_readout_top ==> S_AXI2 @ 0x8001_0000 ==> /dev/uio (aclk)
| (ACLK_RCV; ts <- ts_aclk)
|
+== dbg_aclk_event/data/valid ==> aclk_lite_bridge
    (real events only, rx_usrclk2 -> clk_80m)
                                |
                                v
                                aclk_lite_encoder ==> B10
                                (frame_type=2, 12-byte) (ACLK-Lite)
```

Legend: ==>=data path --==status []=external

2.2 PS software / Redis path

```
/dev/uio (tclk) --> redis_publish.py --src tclk --\
                    (drop UNSYNC ts==0)          >-- RedisSink --> local Redis
/dev/uio (aclk) --> redis_publish.py --src aclk --/    (bg thread)    (KR260: ns)
                                                    |
/dev/uio (wr)   --> wr_time.py arm/status  (arm the WR clock; not published)|
                                                    v
                                                    KR260:tclk / KR260:aclk  (Streams)
                                                    KR260:event:<src>:0x<CODE> (index)
                                                    KR260:status / KR260:watchdog (liveness)
```

External TCLK enters on H12 and is decoded by the single `TCLK_RCV` inside readout #1 (published at `0x8000_0000`) and also fed to the TCLK-to-ACLK encoder. The encoder drives the GT transmitter out the SFP+ cage; an external fiber loop returns it to the SFP+ receiver, where the GT 8b10b-decodes and comma-aligns it and the single `ACLK_RCV` inside readout #2 recovers events (published at `0x8001_0000`). That decoder's tap feeds the ACLK-Lite mirror on B10. A White Rabbit node's 10 MHz + PPS discipline three `wr_timebase` copies so both readouts stamp one `{sec, ns}`; the monitor copy is exposed at `0x8002_0000`. On the PS, one `redis_publish.py` per readout drains its FIFO and writes each event into Redis. (`rtl/aclk_pipeline_bd_top.v`; `vivado/build_aclk_pipeline.tcl`; `deploy/redis_publish.py`).

3. Major Blocks and Responsibilities

3.1 PL blocks (in the bitstream)

Block / module	Responsibility	Clock domain	Source
<code>tclk_readout_top</code> (<code>u_ro_tclk</code>)	Owns the only <code>TCLK_RCV</code> ; decodes H12 biphasic-mark to event byte + strobe; WR-timestamps, buffers, AXI4-Lite (<code>S_AXI</code>)	<code>clk_40m</code> / <code>s_axi_aclk</code>	<code>rtl/aclk_lite/tclk_readout_top.sv</code>
<code>aclk_tclk_encoder</code> (<code>u_enc</code>)	Re-encodes the live TCLK event stream as the 96-bit ACLK frame <code>{0xBC, EVENT, DATA, CRC8}</code> , null-filled, gearboxed 96→16 to GT TX	<code>tx_usrclk2</code> (from <code>clk_40m</code>)	<code>rtl/aclk_gt/aclk_tclk_encoder.v</code>
<code>aclkgt_gt</code> (<code>u_gt</code>)	GTHE4 transceiver: 1.25 Gbps, 8b10b, K28.5 comma, QPLL0, real SFP RX + TX	GT user clocks	<code>vivado/ip/aclkgt_gt/*</code>
RX-recovery FSM	SEARCH/LOCKED/RECOVER self-healing: latches comma-align once locked, re-aligns on sustained byte-align loss	<code>rx_usrclk2</code>	<code>rtl/aclk_pipeline_bd_top.v:305-346</code>
<code>aclk_gt_readout_top</code> (<code>u_ro_aclk</code>)	Owns the only <code>ACLK_RCV</code> ; decodes GT word stream to EVENT/DATA; WR-timestamps, buffers, AXI4-Lite (<code>S_AXI2</code>); taps decoded events	<code>rx_usrclk2</code> / <code>s_axi_aclk</code>	<code>rtl/aclk_gt/aclk_gt_readout_top.sv</code>

Block / module	Responsibility	Clock domain	Source
<code>aclk_lite_bridge</code> (<code>u_bridge</code>)	Filters real events (<code>event[7:0] != 0xFF</code>), crosses <code>rx_usrclk2</code> → <code>clk_80m</code> via async FIFO, drives the encoder when idle	<code>rx_usrclk2</code> → <code>clk_80m</code>	<code>rtl/aclk_lite_bridge.v</code>
<code>aclk_lite_encoder</code> (<code>u_lite</code>)	Free-running biphasic-mark cell engine; emits 12-byte ACLK-Lite frames, idle = continuous 1-cells	<code>clk_80m</code>	<code>rtl/aclk_lite/aclk_lite_encoder.sv</code>
<code>wr_timebase × 2</code> (<code>u_tb_tclk</code> , <code>u_tb_aclk</code>)	One { <code>sec</code> , <code>ns</code> } replica per event domain, both watching the same WR 10 MHz + PPS pins; <code>ts</code> is strictly 0 until armed + locked	<code>clk_40m</code> , <code>rx_usrclk2</code>	<code>rtl/aclk_pipeline_bd_top.v:407-429</code>
<code>wr_timebase_axi</code> (<code>u_tb_axi</code>)	Monitor <code>wr_timebase</code> copy + AXI4-Lite register slave (<code>S_AXI3</code>); arms seconds, reports lock/health	<code>s_axi_aclk</code>	<code>rtl/wr_timebase_axi.sv</code>
<code>aclk_readout_axi</code> / <code>_core</code>	Shared decoder-agnostic timestamping packer + dual-clock FIFO + AXI4-Lite register block (both readouts)	event domain / <code>s_axi_aclk</code>	<code>rtl/aclk_readout/</code>

Note: `global_timebase.v` still exists in the tree but is **not** instantiated by this top; the pipeline replaced the free-running PL tick counter with the WR timebase (grep: `global_timebase` absent from `rtl/aclk_pipeline_bd_top.v`).

3.2 PS software blocks (on the board, not in the bitstream)

Block	Responsibility	Source
<code>readout_common.py</code>	Shared 16-byte-stride register map, mmap open, watchdog-guarded AXI access, <code>drain_events</code> , WR { <code>sec</code> , <code>ns</code> } split/UTC helpers	<code>deploy/readout_common.py</code>
<code>wr_time.py</code>	Arms / monitors the WR timebase over <code>/dev/uio</code> (<code>S_AXI3</code>): <code>status</code> , <code>arm</code> , <code>disarm</code> , <code>clear</code>	<code>deploy/wr_time.py</code>
<code>redis_publish.py</code>	Drains one readout UIO, drops UNSYNC events, builds one Redis record per event, hands it to a <code>RedisSink</code>	<code>deploy/redis_publish.py</code>
<code>redis_sink.py</code> (<code>RedisSink</code>)	Background writer thread: bounded drop-oldest queue → batched <code>XADD</code> / <code>HSET</code> / <code>HINCRBY</code> pipeline, reconnect-with-backoff, status/watchdog liveness	<code>deploy/redis_sink.py</code>
<code>tclk_read.py</code> / <code>aclkgt_read.py</code>	Human console readers of the same FIFOs (<code>--wr</code> prints the WR timeline); alternative to the publisher, not run at the same time on one UIO	<code>deploy/tclk_read.py</code> , <code>deploy/aclkgt_read.py</code>

4. External Interface Summary

Integration-critical ports only; the full port list is in Appendix A. All PL ports are on the top module `aclk_pipeline_bd_top`.

Board pin map (KR260 carrier, PMOD1, all LVCMOS33). Package pins are authoritative (from the XDC); the PMOD-header *position* numbers are what the repo's docs record but the connector-position numbering is documented as ambiguous across sources, so wire by **package pin** and confirm against the carrier silkscreen.

Package pin	PMOD1 pos	Signal	Purpose
H12	1	<code>tclk</code>	TCLK biphasemark input
B10	2	<code>aclk_lite_out</code>	ACLK-Lite Manchester mirror output
E10	3	<code>wr_clk10</code>	White Rabbit 10 MHz reference in
E12	4	<code>wr_pps</code>	White Rabbit PPS in
D11	6	<code>dbg_hb</code>	Readout #1 liveness heartbeat (scope point)

Package pins: `kr260_aclk_pipeline.xdc:15,40,44,49-50`. PMOD positions cross-checked against `deploy/wr.md:9-14` and the carrier pin map; H12/E10/E12=pos 1/3/4 are stated in the XDC comments, B10/D11 positions are from the pinout table and may be mislabeled by connector-position convention.

4.1 Clocks and resets

Signal	Dir	Width	Description
<code>s_axi_aclk</code>	in	1	PS clock for all three AXI slaves + WR monitor; <code>pl_clk0</code> , 100 MHz
<code>clk_80m</code>	in	1	80 MHz: serdec oversample + ACLK-Lite encoder cell clock
<code>clk_40m</code>	in	1	40 MHz: TCLK deserializer + readout #1 event/timestamp domain
<code>freerun_50</code>	in	1	50 MHz free-running clock for the GT reset controller
<code>gt_refclk_p/n</code>	in	1 ea	GT QPLL reference, <code>MGTREFCLK0_224</code> , 156.25 MHz (period 6.400 ns)
<code>s_axi_aresetn</code>	in	1	AXI reset, active-low (<code>pl_clk0</code> domain)
<code>rstn</code>	in	1	PL reset, active-low (<code>peripheral_aresetn</code> , <code>pl_clk0</code>); master reset for all PL logic

Provenance: `s_axi_aclk` / `s_axi_aresetn` / `rstn` from the PS `pl_clk0` / `proc_sys_reset` ;
`clk_80m` / `clk_40m` from `clk_wiz_0` , `freerun_50` from `clk_wiz_freerun`
(vivado/build_aclk_pipeline.tcl:123-174).

4.2 Data path

Signal	Dir	Width	Protocol	Description
<code>tclk</code>	in	1	biphase-mark baseband	TCLK line, LVCMOS33, H12
<code>wr_clk10</code>	in	1	10 MHz CMOS	WR reference clock, async data input, E10
<code>wr_pps</code>	in	1	1 Hz CMOS pulse	WR PPS, phase-aligned to <code>wr_clk10</code> , E12
<code>gt_rxp/rxn</code>	in	1 ea	1.25 Gbps serial	SFP+ RX (looped-back ACLK)
<code>gt_txp/txn</code>	out	1 ea	1.25 Gbps serial	SFP+ TX (re-encoded ACLK)
<code>aclk_lite_out</code>	out	1	biphase-mark baseband	ACLK-Lite mirror, LVCMOS33, B10

Signal	Dir	Width	Protocol	Description
S_AXI (0x8000_0000)	slave	8-bit addr	AXI4-Lite	TCLK readout register block
S_AXI2 (0x8001_0000)	slave	8-bit addr	AXI4-Lite	ACLK readout register block
S_AXI3 (0x8002_0000)	slave	8-bit addr	AXI4-Lite	WR timebase monitor / arm register block

The SFP path is a **physical fiber loopback** on one board (SFP TX port to its own SFP RX port) (rtl/ack_pipeline_bd_top.v:5-13). `wr_clk10` / `wr_pps` are ordinary async LVCMOS33 inputs (2-FF synced per consuming domain), not clock-capable pins; no `create_clock` / `set_input_delay` (kr260_ack_pipeline.xdc:46-50).

4.3 Configuration / control

There are no top-level configuration pins. All runtime control is register-based: the `GT_CTRL` register (`0xF0`) on the **ACLK** slave (S_AXI2), the `FILTER_CFG` register (`0xD0`) on either readout slave, and the **WR SEC_ARM** / **CTRL** registers on S_AXI3. See Section 8.

4.4 Status / error (top-level sideband)

Signal	Dir	Width	Meaning	Recommended handling
<code>sfp_tx_disable</code>	out	1	Driven constant 0 to enable the laser (high/float = laser OFF)	Constrain to Y10; do not repurpose
<code>sfp_tx_fault</code>	in	1	Module TX fault (surfaced in ACLK DEBUG word)	Monitor via DEBUG bit
<code>sfp_rx_los</code>	in	1	Module RX loss-of-signal / no light	Monitor via DEBUG bit
<code>sfp_mod_abs</code>	in	1	Module absent	Monitor via DEBUG bit
<code>dbg_hb</code>	out	1	Readout #1 deep-CDC heartbeat probe pin (D11)	Optional scope point

5. Clock, Reset, and WR Timebase Initialization

Clock domains:

Clock	Freq	Use
<code>pl_clk0</code> (<code>s_axi_ack</code>)	100 MHz	SmartConnect + all three AXI slaves + WR monitor
<code>clk_80m</code>	80 MHz	<code>serdec4_9MHz</code> oversample + ACLK-Lite encoder
<code>clk_40m</code>	40 MHz	TCLK byte framer + readout #1 + WR TCLK replica
<code>gt_refclk</code>	156.25 MHz	GT QPLL reference (free-running at power-up)
<code>tx_usrclk2</code>	62.5 MHz	<code>ack_tclk_encoder</code> framing to GT TX
<code>rx_usrclk2</code>	62.5 MHz	<code>ACLK_RCV</code> + readout #2 + WR ACLK replica

The 62.5 MHz figures are GT-IP user-clock outputs (not asserted numerically in RTL); the WR ACLK replica is parameterized `CLK_PERIOD_DS=160` (16.0 ns = 62.5 MHz), which corroborates the rate (rtl/ack_pipeline_bd_top.v:419-423).

Reset: `rstn` (active-low) is the master PL reset from `peripheral_aresetn`. Per-domain resets are **async-assert** / **sync-deassert**: `ro_rstn` (rx, gated on `rx_active`), `gen_rstn` (tx, gated on `tx_active`), and

`s_axi_aresetn` for the AXI domain. The GT is reset via `gtwiz_reset_all_in = ~rstn` (rtl/aclk_pipeline_bd_top.v:239,287-303).

Clock-domain crossings. Every cross-domain path uses a CDC-safe structure:

- TCLK event `clk_40m` → `tx_usrclk2`: DAVn-toggle 2-FF synchronizer + edge detect (encoder).
- WR arm `cfg s_axi_aclk` → `clk_40m` / `rx_usrclk2`: `cdc_word_pulse` toggle-CDC inside each replica.
- ACLK-back `rx_usrclk2` → `clk_80m`: bridge async FIFO.
- Both readouts' event domain → `s_axi_aclk`: async FIFO (data) + `cdc_gray_count` (counters) + 2-FF (status).

The XDC declares the three clock families (PS, MMCM, GT) mutually asynchronous with one `set_clock_groups` stanza, cutting every cross-family path; each cut path is CDC-safe (kr260_aclk_pipeline.xdc:52-88).

WR timebase behavior (the critical difference from the old design). `ts` is strictly 0 in both readouts until the PS arms a seconds label and a PPS loads it. A zero timestamp means "not WR-synced when stamped" and the publisher **drops** it. Loss of either WR reference (10 MHz or PPS) unlocks all copies and sets a sticky flag; a fresh `arm` is then required. A GT relock (recovery FSM, or `GT_CTRL[24]`) stops `rx_usrclk2` and resets the ACLK replica, so it unlocks; **re-arm after any GT recovery** (rtl/aclk_pipeline_bd_top.v:392-429; deploy/wr.md:42-47).

WR arm timeline (PS side, `wr_time.py`):

```
wr_time.py arm
| wait for a mid-second moment (frac in [0.10, 0.80]) so the write
|   cannot race the PPS boundary
v
write SEC_ARM = floor(now)+1          (UTC label of the NEXT PPS; NTP time)
| HW loads it at that PPS in every timebase copy and locks
v
poll STATUS until locked_tclk & locked_aclk & locked_mon
| verify |HW SEC_NOW - system clock| < 0.5 s
v
ts now advances on the real timeline; publishers begin emitting events
```

GT RX bring-up / recovery timeline:

state	condition	effect
SEARCH	power-up / after RECOVER	comma-align ENABLED
	rx_aligned (>=5 good CRC) -> LOCKED	
LOCKED	normal operation	comma-align LATCHED off
	byteali low >= 512 cyc (~8 us)->RECOVER	(loss window)
RECOVER	hold 512 cycles, then -> SEARCH	decoder reset pulsed

`RECOVER_GT_RESET=0`, so recovery is a soft re-align (decoder + lock reset, **not** the async-FIFO pointers and not a full GT RX datapath reset). A full RX PLL+CDR relock can still be forced at runtime via `GT_CTRL[24]`, but remember it unlocks the ACLK WR replica (rtl/aclk_pipeline_bd_top.v:311-346).

6. Data and Control Flow

Trace one TCLK event from the wire all the way into Redis.

1. **TCLK decode.** `TCLK_RCV` recovers an 8-bit event byte and a one-cycle active-low `DAVn` on `clk_40m`. Readout #1 adapts this as `aclk_valid = ~DAVn`, `aclk_event = {8'h00, DATA}`, `flags = 0x0002` (`is_tclk`), `DROP_NULL=0` (every byte kept, since `0xFF` is a valid TCLK code). It stamps with `ts_tclk` (the WR TCLK replica) and pushes into the FIFO read at `0x8000_0000`.
2. **Re-encode.** The same `dbg_data / dbg_dav` feed `aclk_tclk_encoder` (`tclk_davn = ~dbg_dav`). The encoder keeps a 256-entry per-event-code count RAM, emits an 80-bit payload `{8'h00, event, 32'h0, count}`, wraps it as `{0xBC, payload, CRC8}`, and gearboxes 96→16 to the GT. Between events it emits `0xFF..FF` null frames.
3. **SFP round trip.** The GT transmits at 1.25 Gbps; the fiber loop returns the light; the GT 8b10b-decodes and comma-aligns, producing `rx_data16 + rxctrl2[1:0]` (K flags) on `rx_usrclk2`.
4. **ACLK decode.** `ACLK_RCV` reassembles the 96-bit frame, checks CRC-8, and on `CRC==0` emits `ACLK_EVENT / ACLK_DATA / ACLK_VALID`. Readout #2 keeps `flags=0x0001` (`has_data`), `DROP_NULL=1` (drops `0xFF`-low-byte nulls), stamps with `ts_aclk` (the WR ACLK replica), and publishes at `0x8001_0000`.
5. **ACLK-Lite mirror.** The decoder tap (`dbg_aclk_event/_data/_valid`) feeds `aclk_lite_bridge`, which filters real events (`event[7:0] != 0xFF`), crosses them `rx_usrclk2` → `clk_80m` through a 16-deep async FIFO, and pulses `enc_start` with `frame_type=2` whenever the encoder is idle. `aclk_lite_encoder` serializes a 12-byte biphasemark frame on B10, returning to idle 1-cells between frames.
6. **PS read + publish.** For each readout, `redis_publish.py` polls `STATUS`, reads `EVENT / DATA / TS`, writes `POP`, and unless `ts==0` (UNSYNC, dropped) submits one record to the background `RedisSink`, which writes it to Redis (Section 9). `wr_time.py` on `S_AXI3` must have armed the timebase first, or every event is UNSYNC and nothing is published.

Backpressure / ordering. TCLK events are far slower than the ~12 μ s ACLK-Lite encode time, so the bridge FIFO should not fill; on overflow the event is dropped and counted internally (`dropped_count`, not on a top port). One TCLK event produces exactly one non-null ACLK frame; nulls fill the gaps and are dropped at readout #2. On the PS, the `RedisSink` queue is the second backpressure point: it never blocks the FIFO drain; on a full queue it drops the **oldest** record (counted) so a Redis stall can never wedge the hardware read (`deploy/redis_sink.py:54-73`). Events are paired on the consumer side by event code plus WR time, disambiguated by the per-event `count` in the low 32 bits of `DATA`.

7. Integration Instructions

7.1 Build / instantiation

The reference build (`vivado/build_aclk_pipeline.tcl`) drops the top into a Zynq block design as a module-reference cell; Vivado infers the **three** AXI4-Lite slaves (`S_AXI`, `S_AXI2`, `S_AXI3`) from the `X_INTERFACE` attributes, and a single-clock `SmartConnect` (`NUM_SI=1`, `NUM_MI=3`) fans the LPD master out to all three.


```

set u [create_bd_cell -type module -reference aclk_pipeline_bd_top u_pipeline]
connect_bd_net [get_bd_pins ps/pl_clk0] [get_bd_pins u_pipeline/s_axi_aclk]
connect_bd_net [get_bd_pins rst_pl0/peripheral_aresetn] \
    [get_bd_pins u_pipeline/s_axi_aresetn]
connect_bd_net [get_bd_pins rst_pl0/peripheral_aresetn] [get_bd_pins u_pipeline/rstn]
connect_bd_net [get_bd_pins clk_wiz_0/clk_out1] [get_bd_pins u_pipeline/clk_80m]
connect_bd_net [get_bd_pins clk_wiz_0/clk_out2] [get_bd_pins u_pipeline/clk_40m]
connect_bd_net [get_bd_pins clk_wiz_freerun/clk_out1] [get_bd_pins u_pipeline/freerun_50]
# SmartConnect NUM_SI=1 NUM_MI=3 -> S_AXI, S_AXI2, S_AXI3 ; then assign_bd_address (7.3)
# Build: .\hw.ps1 build -Tcl vivado\build_aclk_pipeline.tcl -Name aclk_pipeline

```

7.2 Parameters

The top has no parameters; it fixes the sub-block parameters internally. The ones worth knowing:

Parameter	Value in pipeline	Effect
ADDR_WIDTH	6 (both readouts)	FIFO depth = $2^6 = 64$ events per readout
AXI_ADDR_W	8 (all three slaves)	8-bit byte-address register space
USE_EXT_TS	1 (both readouts)	Use the shared WR <code>ts_ext</code> , not an internal counter
DROP_NULL	0 (TCLK) / 1 (ACLK)	Keep all TCLK bytes / drop <code>0xFF</code> ACLK nulls
CLK_PERIOD_DS	250 (TCLK) / 160 (ACLK)	WR replica tick in deci-ns: 25.0 ns / 16.0 ns
SAMPLES_PER_CELL	8 (ACLK-Lite enc)	clk_80m cycles per 100 ns biphasic-mark cell

7.3 Required connections and sequencing

1. Provide `s_axi_aclk` = `pl_clk0` (100 MHz), `clk_80m` / `clk_40m` from one MMCM, `freerun_50` from another, and `gt_refclk_p/n` (156.25 MHz).
2. Provide the WR references on E10 (`wr_clk10`) and E12 (`wr_pps`) from a push-pull 3.3V CMOS WR node (PPS phase-aligned to the 10 MHz). Open-drain sources misbehave through the carrier's auto-direction translators.
3. Release `rstn` / `s_axi_aresetn` after PL programming. The device-tree overlay releases `pl_resetn0-3`; without it the fpga-region holds the design in reset and every AXI access bus-errors.
4. Assign the three AXI segments: `0x8000_0000` (S_AXI/TCLK), `0x8001_0000` (S_AXI2/ACLK), `0x8002_0000` (S_AXI3/WR), each range `0x1_0000`. A module-ref slave carries no IP-XACT map, so call bare `assign_bd_address` first to create the segments, then relocate each.
5. Connect the external fiber loop (SFP TX → SFP RX). Wait for GT lock and RX alignment before trusting ACLK events (poll the ACLK slave's `LOCK` and DEBUG-word `rcv_aligned`).
6. On the board: load the overlay, **arm the WR timebase** (`wr_time.py arm`), then start the Redis publishers (Section 9). Until the WR timebase is armed and locked, every event is UNSYNC and nothing publishes.

Use `SmartConnect`, **not** the auto interconnect + protocol-converter path: the latter corrupts AXI4-to-AXI4-Lite read data on this hardware (vivado/build_aclk_pipeline.tcl:126-129).

8. Register Interfaces

8.1 Readout block (both S_AXI and S_AXI2)

Both readouts expose the same `aclk_readout_axi` register block. **Registers are spaced 16 bytes apart, not 4.** On the KR260 LPD path this hand-written slave only returns correct data at 16-byte-aligned offsets (any

offset with `araddr[3:2] != 0` read back 0 on hardware; root cause unpinned, 16-byte spacing sidesteps it) (deploy/readout_common.py:19-23).

Offset	Name	Access	Fields / meaning
0x00	STATUS	RO	bit0 = empty, bit1 = overflow (sticky: an event was lost)
0x10	EVENT	RO	{FLAGS[31:16], EVENT[15:0]} of FIFO head; FLAGS bit0=has_data, bit1=is_tclk
0x20	DATA_HI	RO	DATA[63:32]
0x30	DATA_LO	RO	DATA[31:0]
0x40	TS_HI	RO	WR timestamp seconds (<code>ts[63:32]</code>); 0 = UNSYNC
0x50	TS_LO	RO	WR timestamp nanoseconds (<code>ts[31:0]</code>)
0x60	POP	WO	write any value to pop the head and advance
0x70	EVENT_COUNT	RO	events enqueued (kept)
0x80	NULL_COUNT	RO	null / idle packets dropped (ACLK: 0xFF nulls; TCLK: stays 0)
0x90	ERROR_COUNT	RO	bad-CRC events (ACLK) / new parity errors (TCLK)
0xA0	DEBUG	RO	caller-supplied debug word (differs per slave; see below)
0xB0	HEARTBEAT	RO	free-running event-clock counter (CDC liveness)
0xC0	LOCK	RO	bit0 = MMCM/GT locked (synchronized)
0xD0	FILTER_CFG	WO	{bit8=drop, bits[7:0]=code} set/clear a per-code drop-mask bit
0xE0	FILTERED_COUNT	RO	events dropped by the mask
0xF0	GT_CTRL	RW	GT static control (ACLK slave only; see below)

TS is now a WR {sec, ns} pair, not a free-running tick. `TS_HI` = Unix UTC seconds, `TS_LO` = nanoseconds within the second; the pair is 0/0 while unsynced (rtl/aclk_pipeline_bd_top.v:403-429; deploy/readout_common.py:72-84).

DEBUG word (0xA0) differs per slave. On the TCLK slave it is {sig_err, raw_level, tclk_transitions[29:0]}, a raw-line activity monitor. On the ACLK slave it is a GT-health word {rcv_aligned, byteali, rx_los, notintbl_sticky, disperr_cnt[13:0], tx_fault, mod_abs, recover_cnt[3:0], commadet_cnt[7:0]} (rtl/aclk_pipeline_bd_top.v:389-390).

GT_CTRL (0xF0), ACLK slave only. Reset 0 = normal operation; leave at 0 for default fiber-loopback bring-up. On the TCLK slave the `gt_ctrl` output is unconnected (writes latched but drive nothing).

```

bit0      rxpolarity      bit[4:2] loopback (000=normal,010=near-end PMA)
bit1      txpolarity      bit[8]   RX datapath re-init pulse
bit[13:9] txdiffctrl (0 -> proven default 0x18)
bit[18:14] txpostcursor   bit[23:19] txprecursor
bit[24]    full RX PLL+CDR relock (also unlocks the ACLK WR replica -> re-arm)

```

8.2 WR timebase monitor (S_AXIS3 @ 0x8002_0000)

Same 16-byte stride. Register select = `addr[7:4]` (rtl/wr_timebase_axi.v:8-22).

Offset	Name	Access	Fields / meaning
0x00	STATUS	RO	[0] locked_tclk [1] locked_aclk [2] locked_mon [3] pps_alive [4] clk10_alive [5] arm_pending [8] lost_lock (sticky)
0x10	SEC_ARM	RW	Unix UTC label of the next PPS; the write arms
0x20	SEC_NOW	RO	monitor seconds; the read atomically latches NS_NOW
0x30	NS_NOW	RO	ns latched by the last SEC_NOW read
0x40	PPS_COUNT	RO	PPS edges seen since reset
0x50	CELLS_LAST	RO	10 MHz cells in the last PPS interval (expect 10,000,000)
0x60	CTRL	WO	[0] clear lost_lock sticky [1] broadcast disarm (force unlock)

Arm flow (`wr_time.py`): at a mid-second moment write `SEC_ARM = floor(now)+1`; HW loads it at the next PPS and locks; poll `STATUS` for `locked_tclk` & `locked_aclk` & `locked_mon`; verify `SEC_NOW` vs system clock within 0.5 s. `SEC_NOW/NS_NOW` read 0 while the monitor is unlocked (rtl/wr_timebase_axi.sv:21-22; deploy/wr_time.py:74-99).

8.3 PS read flow (per readout, over `/dev/uio`, offset 0)

Poll `STATUS`; while not empty, read `EVENT`, `DATA_HI`, `DATA_LO`, `TS_HI`, `TS_LO`, then write `POP`; the head is held stable until POP for a consistent snapshot. To drop event codes, write each code with bit8 set to `FILTER_CFG`. This is exactly what `readout_common.read_event / drain_events` do (deploy/readout_common.py:156-164,236-260).

9. Board-side Software and the Redis Interface

The bitstream only buffers events; they become useful through the board-side chain **arm WR** → **drain UIO** → **publish to Redis**. One `redis_publish.py` runs per readout.

9.1 Deploy and run (board)

```
# one-time: install + apply the KR260 Redis settings, restart
sudo apt install -y redis-server python3-redis
cat redis-kr260.conf | sudo tee -a /etc/redis/redis.conf
sudo systemctl restart redis-server && redis-cli ping          # -> PONG

# map /dev/uioN to the readouts, then arm WR before publishing
grep . /sys/class/uio/uio*/name
sudo python3 wr_time.py /dev/uio6 arm                        # WR timebase (S_AXI3)
sudo python3 redis_publish.py /dev/uio4 --src tclk           # TCLK readout
sudo python3 redis_publish.py /dev/uio5 --src aclk           # ACLK readout
```

The `/dev/uioN` indices above are illustrative; resolve them from `/sys/class/uio/*/name`, not by assuming a fixed number (deploy/redis.md:26-33). Nothing publishes until the WR timebase is armed and locked (UNSYNC events, `ts==0`, are dropped) (deploy/redis_publish.py:37-39,81-84).

Publisher options (all optional): `--src` (stream suffix, default `tclk`), `--namespace` (default `KR260`), `--redis-host` (127.0.0.1), `--redis-port` (6379), `--maxlen` (stream cap, default 1,000,000), `--queue-size` (in-process queue, default 100,000) (deploy/redis_publish.py:59-68).

9.2 Redis key schema (namespace `KR260`, matches the Fermilab redis-clock-server convention)

Per published event the sink pipelines three writes:

```
XADD KR260:<src> <event-time-ms>-* { sec, ns, utc, event, data,
                                     is_tclk, has_data, src } MAXLEN ~ <maxlen>
HSET KR260:event:<src>:0x<CODE> { sec, ns, utc, data }
HINCRBY KR260:event:<src>:0x<CODE> count 1
```

Key	Type	Purpose
<code>KR260:tclk</code> , <code>KR260:acclk</code>	Stream	Time-ordered event feed; entry ID is the event time in ms (WR), with a per-stream monotonic guard so a backward WR re-arm cannot make XADD error
<code>KR260:event:<src>:0x<CODE></code>	Hash	Per-event-code index: latest {sec, ns, utc, data} for that code plus a running count
<code>KR260:status</code>	String	<code>1</code> while a publisher is alive; sticky (set on connect, not cleared on stop), do not trust alone
<code>KR260:watchdog</code>	String (TTL)	Refreshed every ~10 s, expires in ~30 s, the authoritative liveness signal

Stream fields are all strings: `sec` / `ns` (WR split), `utc` (ISO-8601, or UNSYNC), `event`, `data`, `is_tclk`, `has_data`, `src` (deploy/redis_publish.py:26-54; deploy/redis_sink.py:100-127).

Consume (read-side examples):

```
redis-cli XLEN KR260:tclk           # climbs while publishing
redis-cli XREVRANGE KR260:tclk + - COUNT 3 # newest 3 (event-time ordered)
redis-cli HGETALL KR260:event:tclk:0x1D  # latest event for code 0x1D + count
redis-cli TTL KR260:watchdog         # counts down from ~30 while alive
```

9.3 Publisher robustness and liveness

- **Never stalls the hardware drain.** The drain thread only `submit()` s to a bounded queue; the separate writer thread talks to Redis. A full queue drops the **oldest** record (`queue_dropped++`) rather than blocking (deploy/redis_sink.py:54-73).
- **Reconnect with backoff.** On any Redis error the writer counts the dropped batch, drops the client, and backs off ~0.5 s (no busy-spin) before reconnecting (deploy/redis_sink.py:146-172).
- **Stats (1 Hz):** `drained` / `published` / `queued` / `queue_dropped` / `redis_dropped` / `reconnects` . `reconnects` counts connect/publish **failures**; if `published` stays 0 while `reconnects` climbs, Redis is unreachable or redis-py is not visible to root (deploy/redis_publish.py:86-92; deploy/redis.md:59-65).
- **Persistence off.** `redis-kr260.conf` sets `save ""` / `appendonly no` and tunes stream nodes; streams are in-memory and start empty on a redis restart (deploy/redis-kr260.conf:5-8).
- **redis-server binds localhost, no auth, keep it single-board** (deploy/redis.md:78).

10. Status, Errors, and Recovery

Indication	Source	Meaning	Recommended response
<code>STATUS.overflow</code> (bit1)	either readout	FIFO overflowed; an event was lost	Read faster / raise <code>ADDR_WIDTH</code> ; sticky until reset

Indication	Source	Meaning	Recommended response
<code>LOCK == 0</code>	either readout	MMCM/GT not locked; event clock may be dead	Fix clocking first; check timebase
<code>HEARTBEAT</code> frozen	either readout	Event-domain clock dead even if <code>LOCK=1</code>	MMCM not producing <code>clk_40m/rx</code> clock
<code>WR STATUS.locked_* == 0</code>	S_AXI3	Timebase not locked; all <code>ts == 0</code> , nothing publishes	Run <code>wr_time.py arm</code> ; check <code>pps_alive/clk10_alive</code>
<code>WR lost_lock == 1</code>	S_AXI3	A WR reference dropped since the last arm	Re- <code>arm</code> ; <code>clear</code> resets the sticky
<code>WR CELLS_LAST</code> far from <code>1e7</code>	S_AXI3	Flaky 10 MHz or PPS line	Fix WR wiring before trusting ns
<code>DEBUG rx_los = 1</code> (ACLK)	GT-health word	No light at SFP RX	Check fiber loop / laser (<code>sfp_tx_disable</code>)
<code>DEBUG rcv_aligned = 0</code> (ACLK)	GT-health word	ACLK decoder not locked	Wait through SEARCH; watch <code>disperr/recover</code>
<code>DEBUG disperr_cnt</code> climbing	GT-health word	8b10b disparity errors on RX	Signal integrity; try <code>GT_CTRL</code> polarity/eq
<code>DEBUG recover_cnt</code> climbing	GT-health word	Recovery FSM re-aligning repeatedly	Link marginal; inspect fiber/GT config
<code>ERROR_COUNT</code> climbing (ACLK)	ACLK_ERROR	Bad-CRC frames	Link errors; correlate with <code>disperr</code>
Redis <code>watchdog</code> TTL expired	Redis	Publisher died / not started	Restart <code>redis_publish.py</code> ; check board
<code>queue_dropped</code> / <code>redis_dropped</code> rising	publisher stats	Redis not keeping up / unreachable	Check <code>redis-cli ping</code> ; <code>redis-py</code> under <code>sudo</code>

Interaction to remember: a GT recovery unlocks the ACLK WR replica, which makes readout #2 stamp UNSYNC and the ACLK publisher go quiet until you re- `arm` (`rtl/aclk_pipeline_bd_top.v:396-399`; `deploy/wr.md:46-47`).

11. Simulation and Validation Notes

The PL was verified sim-first with cocotb 2.0 + Icarus Verilog (`SIM=icarus` default); per project convention tests emit matplotlib occupancy/throughput plots. The Redis path is covered by PC unit tests (`redis-py` stubbed).

Testbench / test	Covers	Source
<code>tb/aclk_pipeline_chain</code>	Full pure-RTL chain: TCLK biphasic → readout #1 + encoder → <code>ACLK_RCV</code> → readout #2, shared-timestamp ordering	<code>tb/aclk_pipeline_chain/</code>
<code>tb/aclk_tclk_encoder_loop</code>	Encoder frames vs golden model; encoder → <code>ACLK_RCV</code> agreement	<code>tb/aclk_tclk_encoder_loop/</code>
<code>tb/aclk_lite_bridge</code>	Bridge + encoder → decode agreement	<code>tb/aclk_lite_bridge/</code>

Testbench / test	Covers	Source
<code>tb/aclk_readout_ext_ts</code>	External-timestamp (WR <code>ts_ext</code>) path through the readout	<code>tb/aclk_readout_ext_ts/</code>
<code>deploy/test_redis_sink.py</code>	RedisSink queue/drop-oldest, batched pipeline, reconnect, watchdog (stub client)	<code>deploy/test_redis_sink.py</code>
<code>deploy/test_redis_publish.py</code>	Record building, UNSYNC drop, field mapping	<code>deploy/test_redis_publish.py</code>
<code>deploy/test_readout_common.py</code>	Register map, event decode, WR split/UTC (off-board)	<code>deploy/test_readout_common.py</code>

The chain sim omits the GT and BRAM IP (pure RTL), so the GT transceiver, SFP electrical behavior, and real fiber are **not** covered in simulation; nor is the live WR 10 MHz/PPS discipline (only the `ts_ext` interface is) (tb layout + rtl instantiation).

12. Limitations, Assumptions, and Open Questions

Operating assumptions

- The WR source drives push-pull 3.3V CMOS with PPS phase-aligned to the 10 MHz; open-drain sources misbehave through the carrier translators (deploy/wr.md:16-18).
- The PS system clock is NTP-disciplined (chrony/timesyncd) so `arm` labels the correct second (deploy/wr_time.py:8-13).

Known limitations

- Publish-side only: no Redis consumer ships here; downstream tools read the streams themselves (deploy/redis.md:4).
- Redis persistence is off; a redis restart clears all streams (deploy/redis-kr260.conf:5-6).
- A publisher-only restart within ~1 s of a backward WR re-arm can have its first events rejected by the stream-ID guard until wall-clock passes the stream top (deploy/redis.md:69-73).

Open questions (need confirmation)

- The PMOD1 header **position** numbers for B10 / D11 disagree across repo docs (connector-position numbering is documented as ambiguous); confirm against the carrier silkscreen before soldering. Package pins are unambiguous.
- End-to-end hardware validation of *this* integrated bitstream (all three slaves + WR + fiber + Redis on one board) is the pending step; the constituent pieces are individually HW-verified.

Appendix A. Full Port List

`aclk_pipeline_bd_top`, all ports.

Signal	Dir	Width	Group	Description
<code>tclk</code>	in	1	Data	TCLK biphasemark input (H12)
<code>wr_clk10</code>	in	1	Data	White Rabbit 10 MHz reference (E10)
<code>wr_pps</code>	in	1	Data	White Rabbit PPS (E12)
<code>gt_refclk_p</code> / <code>gt_refclk_n</code>	in	1 ea	Clock	GT reference clock pair (156.25 MHz, Y6/Y5)
<code>gt_rxp</code> / <code>gt_rxn</code>	in	1 ea	Data	SFP+ RX serial (T2/T1)
<code>gt_txp</code> / <code>gt_txn</code>	out	1 ea	Data	SFP+ TX serial (R4/R3)
<code>freerun_50</code>	in	1	Clock	50 MHz GT reset-controller clock
<code>rstn</code>	in	1	Reset	PL reset, active-low (pl_clk0)
<code>clk_80m</code>	in	1	Clock	80 MHz serdec + ACLK-Lite encoder
<code>clk_40m</code>	in	1	Clock	40 MHz TCLK + readout #1 + WR TCLK replica
<code>aclk_lite_out</code>	out	1	Data	ACLK-Lite biphasemark mirror (B10)
<code>dbg_hb</code>	out	1	Status	Readout #1 heartbeat probe (D11)
<code>sfp_tx_disable</code>	out	1	Sideband	Laser enable (0 = on, Y10)
<code>sfp_tx_fault</code>	in	1	Sideband	Module TX fault (A10)
<code>sfp_rx_los</code>	in	1	Sideband	RX loss-of-signal (J12)
<code>sfp_mod_abs</code>	in	1	Sideband	Module absent (W10)
<code>s_axi_aclk</code>	in	1	Clock	PS clock, all three slaves + WR monitor
<code>s_axi_aresetn</code>	in	1	Reset	AXI reset, active-low
<code>s_axi_*</code> (S_AXI)	slave	8-bit addr	Bus	AXI4-Lite TCLK readout @ 0x8000_0000
<code>s_axi2_*</code> (S_AXI2)	slave	8-bit addr	Bus	AXI4-Lite ACLK readout @ 0x8001_0000
<code>s_axi3_*</code> (S_AXI3)	slave	8-bit addr	Bus	AXI4-Lite WR timebase monitor @ 0x8002_0000

Full AXI4-Lite channel signals (AW/W/B/AR/R) are present on all three slaves per the standard set with [7:0] address and [31:0] data (`rtl/aclk_pipeline_bd_top.v:83-189`).

Appendix B. Source Traceability

Topic	Primary source file(s)	Notes
PL top, three slaves, WR instances, data path	<code>rtl/aclk_pipeline_bd_top.v</code>	Read in full
Build, address map, SmartConnect NUM_MI=3	<code>vivado/build_aclk_pipeline.tcl:122-238</code>	0x8000/0x8001/0x8002_0000
Pin LOCs + async clock groups	<code>constraints/kr260_aclk_pipeline.xdc</code>	H12/B10/E10/E12/D11 + SFP + GT

Topic	Primary source file(s)	Notes
Readout register map (16-byte stride)	<code>deploy/readout_common.py:19-32,156-164</code>	Both readout slaves
WR timebase AXI register map	<code>rtl/wr_timebase_axi.sv:8-22</code>	S_AXI3; STATUS bits, SEC_ARM/NOW
WR arm protocol + gotchas	<code>deploy/wr_time.py</code> , <code>deploy/wr.md</code>	arm/status/disarm/clear
Redis publisher + record building	<code>deploy/redis_publish.py</code>	UNSYNC drop, field/id mapping
Redis writer, queue, liveness	<code>deploy/redis_sink.py</code>	Drop-oldest, batch, watchdog
Redis key schema + runbook	<code>deploy/redis.md</code> , <code>deploy/redis-kr260.conf</code>	KR260 namespace, conf

Generated by the hardware-interface-docs skill. Claims are grounded in the repository source cited inline and in the Source Traceability appendix; items noted as inferred or as open questions should be confirmed against hardware or a design owner before relied upon.